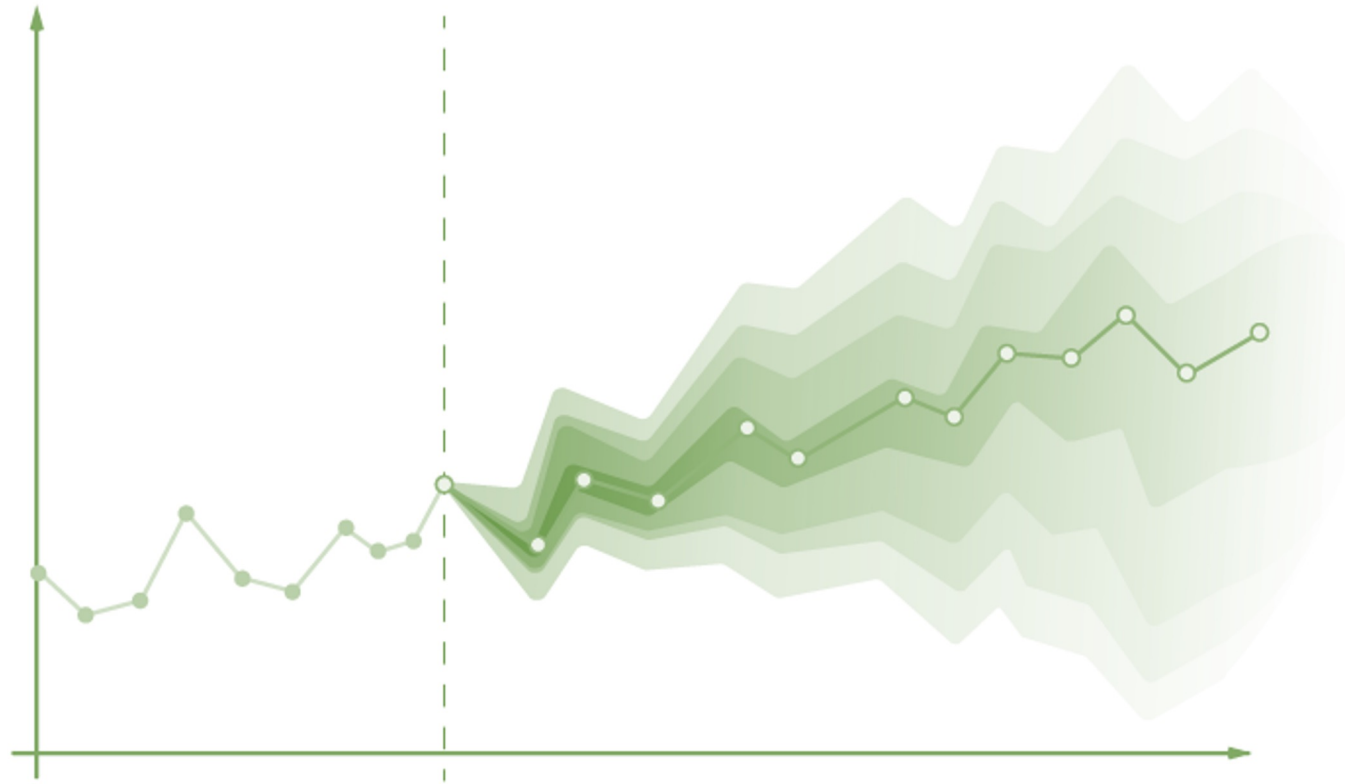




Time Series Forecasting Project

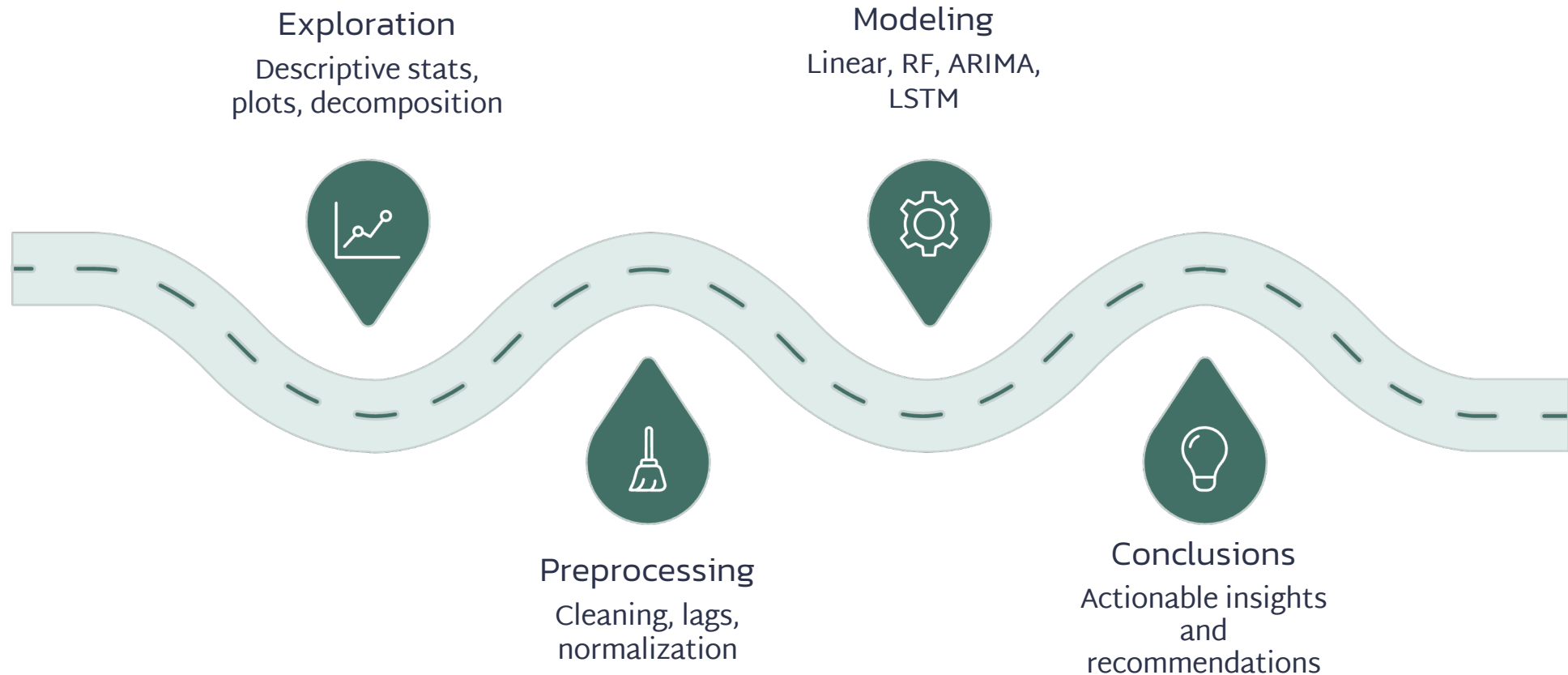
A practical approach to data-driven decision making and analyzing 10 years of daily minimum temperature observations (1981–1990) through a systematic four-phase methodology.

3,650 RECORDS

DAILY MINIMUM TEMPERATURE (°C)

4 PHASES

Four-Phase Methodology



The dataset spans January 1, 1981 to December 31, 1990, with date as the time index and daily minimum temperature (°C) as the continuous target variable.

Chapter 1: Exploratory Data Analysis

Data Loading

Dataset loaded via Google Colab; columns converted to `datetime64` and `float64` for time-series indexing.

Summary Statistics

Mean: **11.18°C** · Median: **11.0°C** · Std Dev: **4.07°C**. Near-identical mean and median confirm a symmetric distribution.

Seasonal Decomposition

Additive model (365-day period) separated trend, seasonal, and residual components — confirming strong annual cycles.

Link to Data and codes in google colab

https://colab.research.google.com/drive/14SuuYfPrnn-swNlyxggNNdzPWJKZ--kU?usp=sharing#scrollTo=K61GTXeJ_yvk

Seasonal Patterns & Correlation Heatmap

The monthly heatmap reveals a clear Southern Hemisphere seasonal pattern:

- **Nov–Feb:** Above-average minimum temperatures (summer)
- **Jun–Aug:** Below-average minimum temperatures (winter)
- **Mar–May & Sep–Oct:** Transition months near the long-term average

Year-to-year variation is visible, with some years showing more pronounced departures from the seasonal norm.

Code Screenshots

Exploratory Data Analysis

```
Importing Pandas to read and make changes in dataset

import pandas as pd

Reading the dataset from Google Drive

from google.colab import drive
drive.mount('/content/drive')
drivefile = '/content/drive/MyDrive/Colab Notebooks/daily-minimum-temperatures-in-me.csv'

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).

df = pd.read_csv(drivefile)

df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3650 entries, 0 to 3649
Data columns (total 2 columns):
#   Column              Non-Null Count  Dtype
---  ---
0    Date                3650 non-null  object
1    Daily minimum temperatures  3650 non-null  object
dtypes: object(2)
memory usage: 57.2+ KB
```

```
Converting it to Datetime and Float64

df['Date'] = pd.to_datetime(df['Date'], format='%m/%d/%Y')
df['Daily minimum temperatures'] = df['Daily minimum temperatures'].str.replace('?', '', regex=False).astype(float)

df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3650 entries, 0 to 3649
Data columns (total 2 columns):
#   Column              Non-Null Count  Dtype
---  ---
0    Date                3650 non-null  datetime64[ns]
1    Daily minimum temperatures  3650 non-null  float64
dtypes: datetime64[ns](1), float64(1)
memory usage: 57.2 KB

df.head()

  Date      Daily minimum temperatures
0  1981-01-01                20.7
1  1981-01-02                17.9
2  1981-01-03                18.8
3  1981-01-04                14.6
4  1981-01-05                15.8
```

```
df.head()

  Date      Daily minimum temperatures
0  1/1/1981                20.7
1  1/2/1981                17.9
2  1/3/1981                18.8
3  1/4/1981                14.6
4  1/5/1981                15.8
```

Finding Mean, Median and Standard Deviation

```
mean = df['Daily minimum temperatures'].mean()
median = df['Daily minimum temperatures'].median()
std = df['Daily minimum temperatures'].std()
```

```
print(f"Mean: {mean}")
print(f"Median: {median}")
print(f"Standard Deviation: {std}")
```

```
Mean: 11.177753424657535
Median: 11.0
Standard Deviation: 4.07183689939719
```

Code Screenshots

Seasonal Patterns & Correlation Heatmap



Chapter 2: Data Preprocessing & Feature Engineering

Missing Values

Forward Fill and Linear Interpolation both yielded zero missing values — confirming the dataset was already complete.

Outlier Detection

IQR Method: 13 outliers · **Z-Score Method:** 10 outliers. IQR is more conservative; Z-score captures only extreme deviations.

Normalization

StandardScaler applied to all numerical features and zero mean, unit variance and preventing scale dominance in distance-sensitive models.

Code Screenshots

Importing Pandas to read and make changes in dataset

```
import pandas as pd
```

Reading the dataset from Google Drive

```
from google.colab import drive
drive.mount('/content/drive')
drivefile = '/content/drive/MyDrive/Colab Notebooks/daily-minimum-temperatures-in-me.csv'
```

↗ Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).

```
df = pd.read_csv(drivefile)
```

```
df.info()
```

↗

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3650 entries, 0 to 3649
Data columns (total 2 columns):
 #   Column                Non-Null Count  Dtype
---  -
 0   Date                  3650 non-null   object
 1   Daily minimum temperatures  3650 non-null   object
dtypes: object(2)
memory usage: 57.2+ KB
```


Code Screenshots

Missing Values

```
Missing Values using Forward Fill

df['Daily minimum temperatures'].fillna(method='ffill', inplace=True)

display(df.isnull().sum())

/tmp/ipython-input-2364923814.py:1: FutureWarning: Series.fillna with 'm
df['Daily minimum temperatures'].fillna(method='ffill', inplace=True)
0
Daily minimum temperatures 0
Month 0
Year 0
DayOfWeek 0
Season 0
Temp_Lag1 0

dtype: int64
```

```
Missing Values using Interpolation

df['Daily minimum temperatures'].interpolate(inplace=True)

display(df.isnull().sum())

0
Daily minimum temperatures 0
Month 0
Year 0
DayOfWeek 0
Season 0
Temp_Lag1 0

dtype: int64
```

Outlier Detection

```
Outlier Detection using IQR Method

Q1 = df['Daily minimum temperatures'].quantile(0.25)
Q3 = df['Daily minimum temperatures'].quantile(0.75)
IQR = Q3 - Q1

lower_bound_iqr = Q1 - 1.5 * IQR
upper_bound_iqr = Q3 + 1.5 * IQR

outliers_iqr = df[(df['Daily minimum temperatures'] < lower_bound_iqr) | (df['Daily minimum temperatures'] > upper_bound_iqr)]

print(f"Outliers detected using IQR method: {len(outliers_iqr)}")

Outliers detected using IQR method: 13
```

```
Outlier Detection using Z-score Method

from scipy.stats import zscore

mean_temp = df['Daily minimum temperatures'].mean()
std_temp = df['Daily minimum temperatures'].std()

# z-scores method
df['Z_Score'] = (df['Daily minimum temperatures'] - mean_temp) / std_temp

z_score_threshold = 3

outliers_zscore = df[(df['Z_Score'] > z_score_threshold) | (df['Z_Score'] < -z_score_threshold)]

print(f"Outliers detected using Z-score method (threshold={z_score_threshold}): {len(outliers_zscore)}")

df.drop('Z_Score', axis=1, inplace=True)

Outliers detected using Z-score method (threshold=3): 10
```

Code Screenshots

Normalization

Normalization (StandardScaler)

```
from sklearn.preprocessing import StandardScaler
import numpy as np
```

```
numerical_cols = df.select_dtypes(include=np.number).columns.tolist()
```

```
scaler = StandardScaler()
```

```
df[numerical_cols] = scaler.fit_transform(df[numerical_cols])
```

```
print("DataFrame after Standardization:")
display(df.head())
```



DataFrame after Standardization:

	Daily minimum temperatures	Month	Year	DayOfWeek	Season	Temp_Lag1	Temp_Lag2
Date							
1981-01-01	2.338883	-1.601508	-1.566699	-0.000548	Summer	NaN	NaN
1981-01-02	1.651139	-1.601508	-1.566699	0.499486	Summer	2.338750	NaN
1981-01-03	1.872199	-1.601508	-1.566699	0.999521	Summer	1.651081	NaN
1981-01-04	0.840583	-1.601508	-1.566699	1.499555	Summer	1.872117	NaN
1981-01-05	1.135330	-1.601508	-1.566699	-1.500651	Summer	0.840613	NaN



Time-Based Feature Engineering

```
# 1. Day of Week (Monday = 0, Tuesday = 1, Wednesday = 2...Sunday=6)
df['DayOfWeek'] = df.index.dayofweek

# 2. Month
df['Month'] = df.index.month
df['Year'] = df.index.year

# 3. Season
def get_season(month):
    if month in [3, 4, 5]:
        return 'Spring'
    elif month in [11, 12, 1, 2]:
        return 'Summer'
    elif month in [9, 10]:
        return 'Autumn'
    else:
        return 'Winter'

df['Season'] = df['Month'].apply(get_season)

# 4. Lag features (e.g., previous day's Consumption)

temp_column = 'Daily minimum temperatures'

if temp_column in df.columns:
    df['Temp_Lag1'] = df[temp_column].shift(1)
    df['Temp_Lag2'] = df[temp_column].shift(7)
else:
    print(f"Error: Temperature column '{temp_column}' not found to create lag features.")

display(df.head())
```

	Daily minimum temperatures	Month	Year	DayOfWeek	Season	Temp_Lag1	Temp_Lag2
Date							
1981-01-01	2.338883	1	1981	3	Summer	NaN	NaN
1981-01-02	1.651139	1	1981	4	Summer	2.338883	NaN
1981-01-03	1.872199	1	1981	5	Summer	1.651139	NaN
1981-01-04	0.840583	1	1981	6	Summer	1.872199	NaN
1981-01-05	1.135330	1	1981	0	Summer	0.840583	NaN

Month / Year

Captures seasonal cycles and long-term trends.

DayOfWeek

Negligible correlation ($r \approx -0.01$) — minimal predictive signal.

Season

Meteorological categories: Spring, Summer, Autumn, Winter.

Temp_Lag1 & Lag2

Strongest predictors: $r \approx 0.77$ (1-day) and $r \approx 0.58$ (7-day lag).

Chapter 3: Model Selection

Four approaches spanning classical statistics to deep learning, evaluated on a chronological 70/15/15 train/val/test split.

Linear Regression Interpretable baseline using lag and time features.	Random Forest Ensemble of decision trees capturing non-linear relationships.
ARIMA (7,0,0) Classical autoregressive model with a 7-day lookback window.	LSTM 50-unit recurrent network; 7-day sliding window; Adam optimizer.

Code Screenshots

Models

```
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
from sklearn.preprocessing import StandardScaler
from statsmodels.tsa.arima.model import ARIMA
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense
from tensorflow.keras.optimizers import Adam
```

```
df = df.dropna()
```

```
# Prepare features and target
X = df[['Temp_Lag1', 'Temp_Lag2']]
y = df['Daily minimum temperatures']
```

```
# Split data: 70% train, 15% validation, 15% test
X_temp, X_test, y_temp, y_test = train_test_split(X, y, test_size=0.15, random_state=42, shuffle=False)
X_train, X_val, y_train, y_val = train_test_split(X_temp, y_temp, test_size=0.176, random_state=42, shuffle=False)
```

```
# Linear Regression
lr = LinearRegression()
lr.fit(X_train, y_train)
y_pred_linear = lr.predict(X_test)
```

```
#RandomForestRegressor
rf.fit(X_train, y_train)
y_pred_test_rf = rf.predict(X_test)
```

```
# Evaluate models
models = {'Linear Regression': lr, 'Random Forest': best_rf}
results = {}
```

```
for name, model in models.items():
    y_pred = model.predict(X_test)
    mse = mean_squared_error(y_test, y_pred)
    mae = mean_absolute_error(y_test, y_pred)
    r2 = r2_score(y_test, y_pred)
    results[name] = {'MSE': mse, 'MAE': mae, 'R2': r2}
```

```
# ARIMA model
arima_train = y_train
arima_model = ARIMA(arima_train, order=(7, 0, 0))
arima_fit = arima_model.fit()
arima_forecast = arima_fit.forecast(steps=len(y_test))
y_pred_arima = arima_forecast
```

```
# LSTM model
import numpy as np
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense
from tensorflow.keras.optimizers import Adam
from sklearn.preprocessing import StandardScaler

def create_lstm_data(data, lookback=7):
    X, y = [], []
    for i in range(lookback, len(data)):
        X.append(data[i-lookback:i])
        y.append(data[i])
    return np.array(X), np.array(y)

# Scale data for LSTM
scaler = StandardScaler()
temp_scaled = scaler.fit_transform(df[['Daily minimum temperatures']])

# Create sequences to include lag 7
lookback = 7
X_lstm, y_lstm = create_lstm_data(temp_scaled.flatten(), lookback)

# Split LSTM data
train_size = int(len(X_lstm) * 0.7)
val_size = int(len(X_lstm) * 0.15)

X_lstm_train = X_lstm[:train_size]
y_lstm_train = y_lstm[:train_size]
X_lstm_val = X_lstm[train_size:train_size + val_size]
y_lstm_val = y_lstm[train_size:train_size + val_size]
X_lstm_test = X_lstm[train_size + val_size:]
y_lstm_test = y_lstm[train_size + val_size:]

X_lstm_train = X_lstm_train.reshape(X_lstm_train.shape[0], lookback, 1)
X_lstm_val = X_lstm_val.reshape(X_lstm_val.shape[0], lookback, 1)
X_lstm_test = X_lstm_test.reshape(X_lstm_test.shape[0], lookback, 1)
```

Code Screenshots

Models

```
# Build LSTM model
lstm_model = Sequential([
    LSTM(50, activation='relu', input_shape=(lookback, 1)),
    Dense(1)
])

lstm_model.compile(optimizer=Adam(learning_rate=0.001), loss='mse')

# Train LSTM
history = lstm_model.fit(
    X_lstm_train, y_lstm_train,
    epochs=50,
    batch_size=32,
    validation_data=(X_lstm_val, y_lstm_val),
    verbose=0
)

# Predict with LSTM
y_pred_lstm_scaled = lstm_model.predict(X_lstm_test)
y_pred_lstm = scaler.inverse_transform(y_pred_lstm_scaled).flatten()
```

```
# Evaluation function
def evaluate_model(y_true, y_pred, model_name):
    mse = mean_squared_error(y_true, y_pred)
    mae = mean_absolute_error(y_true, y_pred)
    r2 = r2_score(y_true, y_pred)
    return {
        'Model': model_name,
        'MSE': mse,
        'MAE': mae,
        'R2': r2
    }

# Evaluate all models
results = []

# Linear Regression
if 'y_pred_linear' in locals():
    results.append(evaluate_model(y_test, y_pred_linear, 'Linear Regression'))
else:
    print("Warning: y_pred_linear is not defined. Skipping Linear Regression evaluation.")

# Random Forest
if 'y_pred_test_rf' in locals():
    results.append(evaluate_model(y_test, y_pred_test_rf, 'Random Forest'))
elif 'y_pred_rf' in locals():
    print("Warning: y_pred_test_rf is not defined. Using y_pred_rf for Random Forest evaluation.")
    results.append(evaluate_model(y_test, y_pred_rf, 'Random Forest'))
else:
    print("Warning: Random Forest predictions (y_pred_test_rf or y_pred_rf) are not defined. Skipping Random Forest evaluation.")

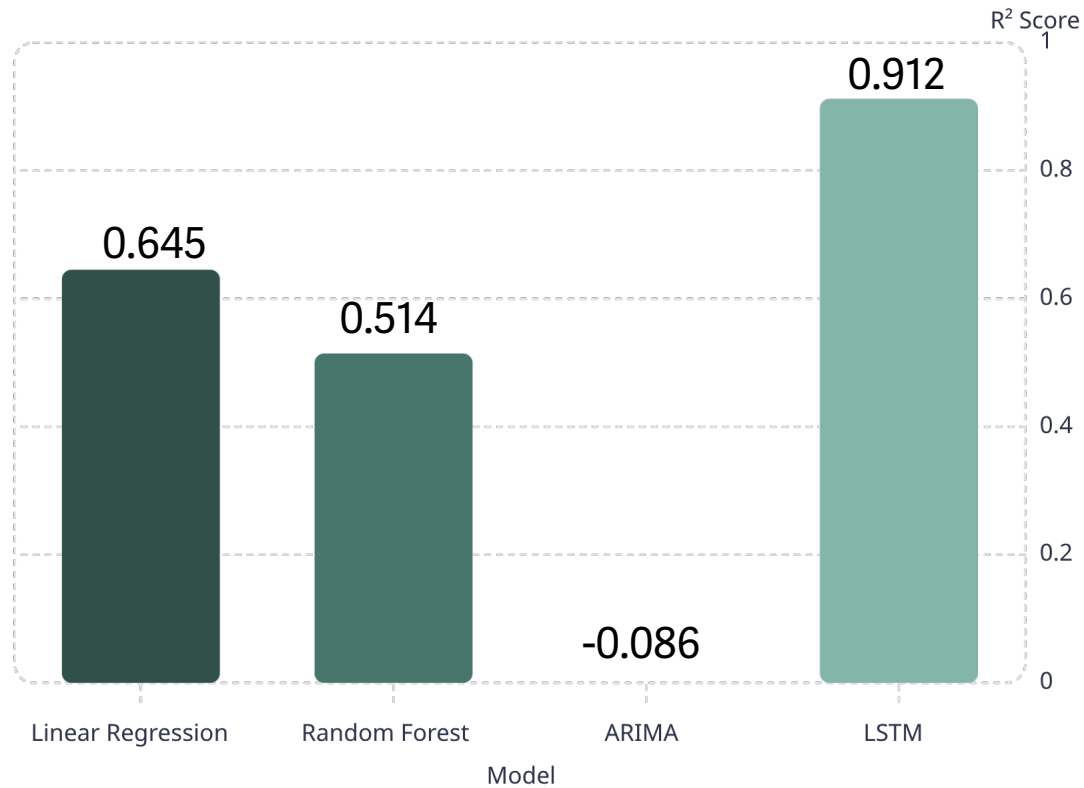
# ARIMA
if 'y_pred_arima' in locals():
    min_len = min(len(y_test), len(y_pred_arima))
    results.append(evaluate_model(y_test[:min_len], y_pred_arima[:min_len], 'ARIMA'))
else:
    print("Warning: y_pred_arima is not defined. Skipping ARIMA evaluation.")

# LSTM
if 'y_pred_lstm' in locals():
    lstm_test_length = len(y_pred_lstm)
    results.append(evaluate_model(y_test[:lstm_test_length], y_pred_lstm, 'LSTM'))
else:
    print("Warning: y_pred_lstm is not defined. Skipping LSTM evaluation.")

# Create results dataframe
results_df = pd.DataFrame(results)
print(results_df)
```

	Model	MSE	MAE	R2
0	Linear Regression	0.325355	0.448498	0.645163
1	Random Forest	0.445238	0.530740	0.514417
2	ARIMA	0.995703	0.827056	-0.085928
3	LSTM	0.080725	0.218314	0.912079

Model Performance Comparison



Key Findings

- **LSTM:** MAE = 0.218, $R^2 = 0.912$ (Best overall Strong sequential pattern capture.)
- **Linear Regression:** $R^2 = 0.645$ (highly effective given informative lag features.)
- **Random Forest:** $R^2 = 0.514$ (default config under-leverages non-linear capability.)
- **ARIMA:** $R^2 = -0.086$. (performed worse than predicting the global mean; suboptimal parameters.)

Hyperparameter Tuning & Final Results

Random Forest GridSearchCV

3-fold cross-validation over `n_estimators`, `max_depth`, and `min_samples_split`.

📄 **Best Parameters:** `n_estimators=200`,
`max_depth=10`, `min_samples_split=10`

Final Test Set Comparison

Model	MSE	MAE	R ²
Linear Regression	0.3254	0.4485	0.6452
Tuned Random Forest	0.3487	0.4669	0.6197

Tuning improved Random Forest marginally, but Linear Regression retained the edge on R² — validating that well-engineered features can outperform complex models.

Code Screenshots

Hyperparameter Tuning

Hyperparameter Tuning

```
from sklearn.model_selection import GridSearchCV

#parameter grid for hyperparameter tuning
param_grid = {
    'n_estimators': [50, 100, 200],
    'max_depth': [None, 10, 20],
    'min_samples_split': [2, 5, 10]
}

grid_search = GridSearchCV(estimator=rf, param_grid=param_grid,
                           scoring='neg_mean_squared_error', cv=3, n_jobs=-1)

grid_search.fit(X_val, y_val)

print("Best parameters found by GridSearchCV:")
print(grid_search.best_params_)

best_rf_model = grid_search.best_estimator_

Best parameters found by GridSearchCV:
{'max_depth': 10, 'min_samples_split': 10, 'n_estimators': 200}
```

Code Screenshots

Visualisations

```
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
import matplotlib.pyplot as plt
import seaborn as sns

if 'lr' in locals() and 'X_test' in locals() and 'y_test' in locals():
    y_pred_linear_test = lr.predict(X_test)
    mse_linear_test = mean_squared_error(y_test, y_pred_linear_test)
    mae_linear_test = mean_absolute_error(y_test, y_pred_linear_test)
    r2_linear_test = r2_score(y_test, y_pred_linear_test)
    print("Linear Regression Evaluation on Test Set:")
    print(f" MSE: {mse_linear_test:.4f}")
    print(f" MAE: {mae_linear_test:.4f}")
    print(f" R2: {r2_linear_test:.4f}")
else:
    print("Linear Regression model or test data not available for evaluation.")

print("-" * 30)

if 'best_rf_model' in locals() and 'X_test' in locals() and 'y_test' in locals():
    y_pred_tuned_rf_test = best_rf_model.predict(X_test)
    mse_tuned_rf_test = mean_squared_error(y_test, y_pred_tuned_rf_test)
    mae_tuned_rf_test = mean_absolute_error(y_test, y_pred_tuned_rf_test)
    r2_tuned_rf_test = r2_score(y_test, y_pred_tuned_rf_test)
    print("Tuned Random Forest Evaluation on Test Set:")
    print(f" MSE: {mse_tuned_rf_test:.4f}")
    print(f" MAE: {mae_tuned_rf_test:.4f}")
    print(f" R2: {r2_tuned_rf_test:.4f}")
else:
    print("Tuned Random Forest model or test data not available for evaluation.")

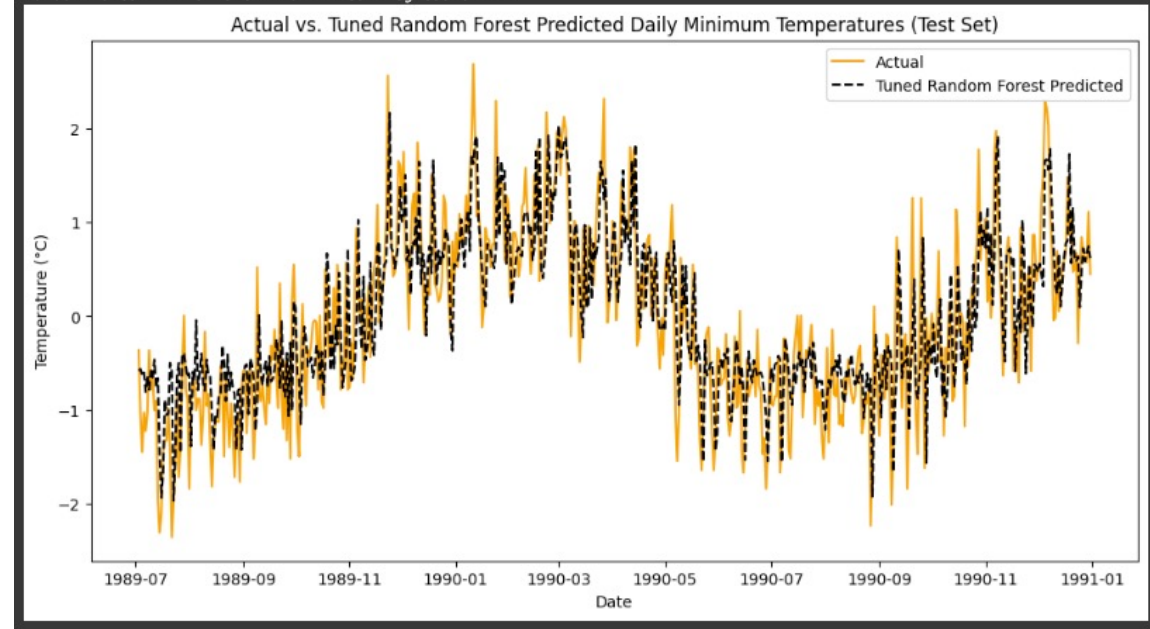
print("-" * 30) # Separator
print("Comparison:")
if 'mse_linear_test' in locals() and 'mse_tuned_rf_test' in locals():
    print(f"Random Forest MSE is {'lower' if mse_tuned_rf_test < mse_linear_test else 'higher'} than Linear Regression MSE.")
    print(f"Random Forest MAE is {'lower' if mae_tuned_rf_test < mae_linear_test else 'higher'} than Linear Regression MAE.")
    print(f"Random Forest R2 is {'higher' if r2_tuned_rf_test > r2_linear_test else 'lower'} than Linear Regression R2.")
else:
    print("Evaluation metrics not available for comparison.")

if 'y_test' in locals() and 'y_pred_tuned_rf_test' in locals():
    plt.figure(figsize=(12, 6))
    plt.plot(y_test.index, y_test.values, label='Actual', color='orange')
    plt.plot(y_test.index, y_pred_tuned_rf_test, label='Tuned Random Forest Predicted', color='black', linestyle='--')
    plt.title('Actual vs. Tuned Random Forest Predicted Daily Minimum Temperatures (Test Set)')
    plt.xlabel('Date')
    plt.ylabel('Temperature (°C)')
    plt.legend()
    plt.show()
else:
    print("Test data or tuned Random Forest predictions not available for visualization.")
```

Linear Regression Evaluation on Test Set:
MSE: 0.3254
MAE: 0.4485
R2: 0.6452

Tuned Random Forest Evaluation on Test Set:
MSE: 0.3487
MAE: 0.4669
R2: 0.6197

Comparison:
Random Forest MSE is higher than Linear Regression MSE.
Random Forest MAE is higher than Linear Regression MAE.
Random Forest R2 is lower than Linear Regression R2.



Code Screenshots

Visualisations

```
import matplotlib.pyplot as plt
import seaborn as sns

if 'results_df' not in locals():
    print("Error: results_df DataFrame is not available for plotting.")
else:

    metrics_to_plot = ['MSE', 'MAE', 'R2']
    results_melted = results_df.melt(id_vars='Model', value_vars=metrics_to_plot,
                                     var_name='Metric', value_name='Value')

    # Create separate plots for each metric
    fig, axes = plt.subplots(1, 3, figsize=(16, 5))
    fig.suptitle('Model Performance Comparison', fontsize=16, fontweight='bold')

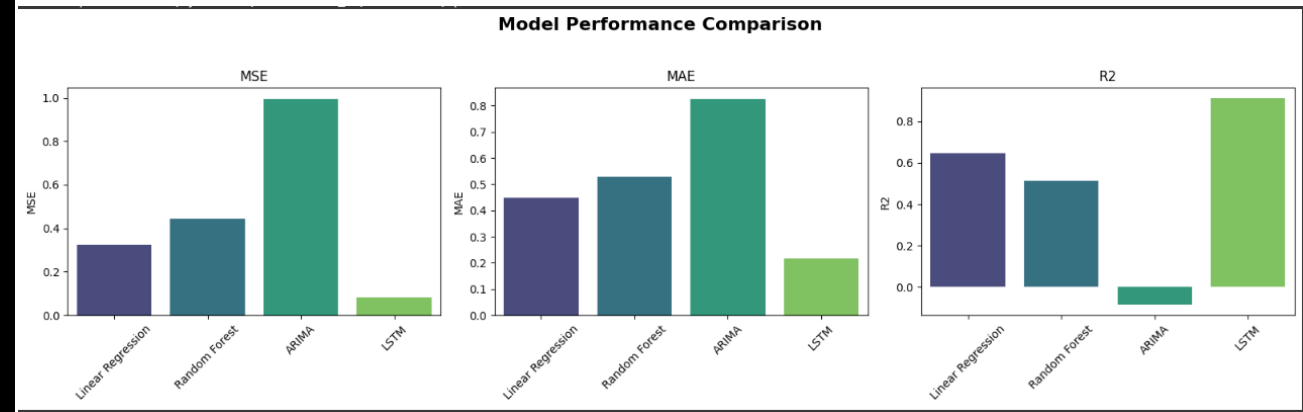
    for i, metric in enumerate(metrics_to_plot):

        metric_df = results_melted[results_melted['Metric'] == metric]

        sns.barplot(x='Model', y='Value', data=metric_df, ax=axes[i], palette='viridis')

        axes[i].set_title(metric)
        axes[i].set_ylabel(metric)
        axes[i].tick_params(axis='x', rotation=45)
        axes[i].set_xlabel('')

    plt.tight_layout(rect=[0, 0, 1, 0.95])
    plt.show()
```



Conclusions & Business Recommendations



Energy & Utilities

Dynamic pricing 24–48h ahead; grid load balancing using 7-day forecasts.



Infrastructure

Predictive maintenance scheduling during forecast periods of optimal weather.



Agriculture

Frost risk alerts, irrigation scheduling, and harvest planning.



Retail & Supply Chain

Inventory optimization for weather-sensitive products.



Tourism & Events

Weather-informed venue and staffing decisions for outdoor events.



Future Work: Auto-ARIMA tuning, multivariate LSTM with exogenous variables, Prophet for seasonality, and rolling-window cross-validation.